

SPACE

SPACE (*student proposal*)

Assessment: ✓ Fit · **Category:** Simulator · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): *SPACE* is a **web application for large-scale events** (conventions, expos) that helps planners **design venues while accounting for crowd movement and emergency access**. Planners generate a **3D floor plan**, place furnishings (booths, signage), and define entry/exit points; the app **simulates crowd movement** and surfaces problems (movement bottlenecks, poorly-placed signage), so planners can adjust the layout. Named load-bearing C++: a **fixed-timestep crowd-simulation loop** (updates attendee movement, detects bottlenecks + restricted-access areas); an **ECS** (DOTS-like, efficient data layout); **agent grouping for pathfinding** (group attendees to cut cost); and **event-driven pathfinding** (recompute only when needed, not every frame). Named technical features: **floor-plan importing** into an editable layout; an **interactive 3D venue editor** (place furnishings, entry/exit); **crowd simulation + overcrowding heatmap**; and **emergency-scenario / evacuation simulation**. Tools: a **file-import system** (pdf/png/jpeg → canvas layer with reference scale); a **3D spatial editor** (drag-and-drop, scaling/rotation, a 3D grid for precise placement + measurement); and a **real-time crowd-simulation tool** (emergency scenarios — density, pathfinding, evacuation routes). Design features: a 3D interactable floor plan; a real-world measurement + grid system; a drag-and-drop 3D basic-shapes panel; and simulation flow analysis. Team: **6 CS + 1 Design + 0 Art**.

Note (category): *SPACE* spans a 3D authoring editor and density visualization, but its technical spine is a **real-time agent-based crowd/evacuation simulation** — so it is assessed as a **Simulator** (the editor sets up the run; the heatmap reads it out). This lands squarely on the Simulator pool (agent behaviour, movement/collision-avoidance, scenario editor, advanced visualization, evacuation behaviour).

Note (module completeness): the app needs to **save/load venue plans, scenarios and run results** — use the module's external DB (Firebase, or the team's Linux server) for this; set the seam up + basic-test it by M2. *Platform:* SPACE is explicitly a **web application** for planners at a desk, so the Main target is the **web build** (browser, HTML/JS/CSS front-end + a C++ engine compiled to **WASM via Emscripten**, rendering through WebGL/OpenGL ES) — there is no native-mobile target, so goals below carry the **Web build** label only.

References:

- [Oasys MassMotion](#) — pedestrian simulation & crowd modelling software (the industry benchmark: a 3D venue authoring + pedestrian-flow simulation tool that surfaces congestion hot-spots as heatmaps and validates fire egress/evacuation for stadia and events — the venue-design + crowd-sim + heatmap + evacuation precedent)
- [Social force model](#) — Wikipedia (the foundational pedestrian-dynamics model: attendees are agents driven by goal-seeking + repulsive social/obstacle forces, which is how bottlenecks and congestion emerge — the crowd-movement algorithm precedent)
- [An evacuation simulator for pedestrian dynamics based on the Social Force Model \(arXiv\)](#) (a worked evacuation simulator: agents egress a floor plan under the social-force model, exactly the emergency/evacuation-scenario feature — the evacuation-simulation precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	SPACE
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	♦
Networking System	●

How each requirement applies:

Platform fit: **Main:** Web build — a browser app (HTML/JS/CSS front-end) with the C++ simulation engine compiled to **WASM via Emscripten**, rendering through WebGL/OpenGL ES; desktop-first (planners at a workstation), responsive web-on-mobile optional. No native-mobile target. The PC is also the dev/tools vehicle. **Extended:** Firebase (or the team's Linux server, set up by M2, full use T2) to store venue plans, scenarios and run results.

Front end: A browser app — the 3D venue editor (grid canvas, drag-and-drop furnishings/booths/signage, entry/exit markers, scale + measurement), the simulation view (attendees moving) and the overcrowding heatmap + flow-

analysis panels. **Languages:** Web — HTML/JS/CSS shell + the C++ engine as WASM (Emscripten), WebGL/OpenGL ES for the 3D scene and the 2D heatmap; some panels are plain web UI.

Back end: The real-time C++ crowd-simulation engine — the fixed-timestep loop over an ECS (DOTS-like), agent grouping + event-driven pathfinding, movement + local collision-avoidance producing emergent bottlenecks, and the evacuation-scenario model — compiled to WASM and driving the browser view; plus a store (Firebase/server) for plans and run results. **Languages:** C++ (engine → WASM); Firebase/server on the Extended tier.

Backend scope: Client-side WASM engine now, cloud-persisted (T2) — the plan/scenario/results store set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ fixed-timestep loop is the heart — it advances every attendee-agent (goal-seeking + collision-avoidance), groups agents for cheaper pathfinding, recomputes paths only on demand (event-driven), and continuously flags bottlenecks + restricted-access areas.
- *Data-driven:* the venue (walls, booths, signage, entry/exit), the agent populations/parameters, and the scenarios (normal vs emergency) are all data — a new layout or scenario is a data change (authored in the editor, JSON round-trip).
- *2D:* the top-down agent visualization and the **overcrowding heatmap / density layers** satisfy the 2D gate (custom-rendered, no chart libs). *3D (♦, strong-students tier):* the 3D venue editor + 3D floor plan is the ambitious layer — recommended to keep the **simulation itself 2D/top-down** (which fully satisfies the gate) and treat 3D as a visualization/editing layer on top, so the 3D scope can flex without risking the core sim.
- *Front-end + Back-end:* a genuine split — a web/JS front-end over a C++/WASM engine; save/load + results storage is **Application Communication** ✓.
- *Custom tools:* three — the **floor-plan import system** (pdf/png/jpeg → scaled canvas layer), the **3D spatial editor** (drag-drop, scale/rotate, measurement grid), and the **crowd-simulation/emergency-scenario tool**.
- *Networking:* a single-planner tool — no real-time multiplayer, so a Networking System is optional (●). *External DB:* plans + scenarios + run results (add per the module). *Auth » / Analytics »:* planner accounts + aggregate run metrics are natural but module-deferred to T2 (●).

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Simulator pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the fixed-timestep crowd loop + the ECS.

Tech & dependencies

- **Tech: Web build:** A fixed-timestep C++ loop over an ECS (DOTS-like, contiguous storage) advancing attendee-agents and detecting bottlenecks/restricted-access, compiled to WASM (Emscripten) with an HTML/JS shell; C++17/20, CMake, Git.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; behaviour (M1G5), movement (M1G7) and the visualization (M1G4) run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** An ECS/data model with the venue (walls, booths, signage, entry/exit), agent populations + parameters, and scenarios (normal/emergency) in hot-reloadable JSON — a new layout or scenario is a data change.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (the 3D editor), M2G6 (advanced editor + import), M1G5/M1G7 (agent + movement data).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui/overlay inspectors (agent state, group membership, path-recompute events), Tracy profiling of a crowded venue to prove the fixed step holds, Catch2 tests (determinism, bottleneck detection, data loading), on a GitHub Actions Windows+Linux matrix (WASM build check).
- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the scale work M2G1 validates.

- **G4 2D Visualization of the Simulated World** — the top-down venue + moving attendees (the base view the heatmap extends).

Tech & dependencies

- **Tech: Web build:** A renderer (WebGL/OpenGL ES via Emscripten) drawing the venue, the attendee-agents moving, entry/exit and early density shading, plus a camera; GLSL, FreeType for labels.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** M2G4 (the overcrowding heatmap/density layers); the editor shares this renderer.

- **G5 Agent / Entity Behavior System** — attendee agents (grouping + event-driven decisions).

Tech & dependencies

- **Tech:** Goal-seeking agents (to a booth/exit) with FSM/decision rules, ****agent grouping**** to cut pathfinding cost, and ****event-driven pathfinding**** (recompute only on layout/goal change), all custom C++; a social-force-style steering model produces emergent congestion.
- **Prerequisites:** G1 (tick), G2 (agent/venue data).
- **Feeds into:** M2G13 (evacuation/panic behaviour); the bottleneck pillar.

- **G7 Movement & Collision-Avoidance Core** — kinematics + local avoidance (where bottlenecks form).

Tech & dependencies

- **Tech:** A C++ integrator + broadphase (uniform grid / quadtree) with local collision-avoidance (social-force / velocity-obstacle style) so agents queue, jostle and jam realistically at chokepoints; GLM math, deterministic.
- **Prerequisites:** G1(loop & tick), G2 (venue geometry).
- **Feeds into:** The bottleneck/overcrowding detection; M2G13 (evacuation).

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — scale the sim + validate the WASM build.

Tech & dependencies

- **Tech: Web build:** Memory pools, cache-friendly ECS iteration, threading where the browser allows (Web Workers / WASM threads), Tracy profiling of a large crowd, sanitizers on the native dev build, and

Emscripten→WASM validation in the browser; the Firebase/Linux-server store set up (account + software + basic connectivity test) by M2.

- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The web deliverable + a smooth large-venue simulation.

- **G2 ★ Content Editor — Core (3D Venue / Scenario Editor)** — place furnishings, define entry/exit, the measurement grid.

Tech & dependencies

- **Tech:** A web/JS front-end (or Dear ImGui in the dev build) over the shared renderer: drag-and-drop booths/signage, mark entry/exit, snap to a real-world-scaled grid, and edit scenarios, with round-trip JSON, live preview and undo/redo.
- **Libraries/Software:** HTML/JS/CSS + WASM bindings (or Dear ImGui), nlohmann/json, the WebGL/OpenGL ES renderer.
- **Prerequisites:** G2 (venue/scenario data), M1G4 (renderer).
- **Feeds into:** All venue content; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, a reproducible Emscripten→WASM build in CI, tests (determinism, bottleneck detection, import round-trip, data loading) and packaging via GitHub Actions.
- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

- **G4 Visualization — Advanced (Overcrowding Heatmap & Density)** — heatmaps, density layers, time scrubbing.

Tech & dependencies

- **Tech: Web build:** WebGL/GLSL shaders + framebuffers rendering an occupancy/overcrowding heatmap and density layers over the venue, with a time-scrub to replay the run and a legend; the "simulation flow analysis" read-out.
- **Prerequisites:** M1G4 (2D visualization), M2G1 (optimization).
- **Feeds into:** The bottleneck/flow-analysis showcase; layout-decision support.

- **G6 Scenario Editor — Advanced (Drag-Drop, Scale/Rotate, Measurement + Floor-Plan Import)** — the full spatial editor + the import system.

Tech & dependencies

- **Tech:** Advanced editor features — drag-drop with scaling/rotation, precise grid placement + real-world measurement, and a ****floor-plan import**** pipeline (pdf/png/jpeg → a scaled reference canvas layer to trace/build over); live preview binding, deep undo/redo.
- **Prerequisites:** M2G2 (editor core), G2 (data model).
- **Feeds into:** Fast venue authoring; the custom-tools showcase.
- **G13 Behavior II — Panic Models & Evacuation** — the emergency-scenario / evacuation simulation.

Tech & dependencies

- **Tech:** Adaptive/panic agent models for emergencies (raised urgency, exit-seeking, herding, reduced spacing), evacuation-route resolution to the defined exits, and egress-time + blockage metrics; custom C++ over the M1 behaviour/movement cores.
- **Prerequisites:** M1G5 (behaviour), M1G7 (movement/avoidance).
- **Feeds into:** The emergency-access safety story; the evacuation showcase.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 0 Art (8) — **as proposed**, and defensible: a spatial-planning/simulation **tool** has a **light art surface** (UI, iconography, the 3D primitive/furnishing look, the heatmap palette), so 0 artists is far less risky here than for an asset-heavy game — the single designer can carry the **D11 conditional** (visual identity + in-app art quality) alongside the tool/UX design. That said, a **solo designer + 0 artists** is still the case the design pool flags ("must ask the instructor which conditional bends", see [design-goals-pool.pdf](#)), and the design load here is real (the planner UX, the analysis presentation, the crowd-model realism and scenario design), so a **2nd designer is recommended** if the team can — one owns the tool/interaction design, the other the D11 art surface + the flow-analysis presentation. *Scope note (advisory):* a WASM crowd-sim engine (ECS + grouping + event-driven pathfinding + collision-avoidance) + a 3D editor + import + heatmaps + evacuation is a **large, genuinely research-flavoured CS scope** — a 2D top-down sim of one venue with basic bottleneck detection + a simple editor is the M1/M2 floor; the 3D editor, floor-plan import, overcrowding heatmap and evacuation scenarios are the stretch. **Keep the simulation 2D** (satisfies the 2D gate) and treat 3D as the strong-students visualization/editing layer — this is the single biggest scope-risk lever.

Track goals — suggested Design / UI-UX / TEAM picks for this project (no Art track as proposed — 0 artists, so the light Art surface is carried by the Design-owned D11 conditional); deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the planner's problem — designing safe, high-flow event venues; the plan → simulate → read bottlenecks/heatmap → adjust loop; the emergency/evacuation value; a primary benchmark e.g. MassMotion, and a Singapore benchmark — e.g. a Marina Bay Sands / Expo / Singapore EXPO planning context)
- M1D2 Core-Loop Interactive Prototype (a clickable prototype of the editor → run → analysis flow, proving the planner UX before the engine is deep) · Figma/Unity for the prototype

- M1D11 Art Proof of Concept *(conditional — mandatory for 0-artist teams; Design-owned)* — the tool's visual identity: UI system, iconography, the 3D primitive/furnishing look and the heatmap palette; 2 mockups.

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D11 Art Implementation Quality *(conditional — mandatory for 0-artist teams; Design-owned)* — the UI, editor chrome, 3D asset look and heatmap styling implemented in-engine, no placeholders.

- **UI/UX M1:**

- M1U1 ★ UI Standards & Wireframe Baseline — standards for the editor (tool palettes, property inspectors, the measurement grid), the run controls and the analysis panels, plus a wireframe baseline; this is a **data-dense professional tool**, so information density + clarity are the priority.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — the editor, run controls, heatmap/flow-analysis and import flow navigable unaided in the browser.
- M2U2 ★ UX Validation & Final Polished UI — validated with target users (planners / people who lay out spaces), polished to the "A" standard in the web build.

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial audio from the week-5 needs list present & working (UI feedback, the emergency-alarm/alert cue for evacuation scenarios, run start/stop), suitable and correctly triggered — modest for a tool, but the evacuation alarm is a real design beat.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable playback.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — the tool's audio set (UI + alert/alarm + run cues), suited to a professional desktop tool.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the playback the build needs (multi-channel; a distinct emergency-alarm cue).

- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the 3D editor + import + the crowd-sim/evacuation tool), prototype demo, team post-mortem (wk14)
-

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*